

Cozy Comfort - Task 3, 4 and 5 Answers

3) Testing Scope and Coverage (LO3)

3.1 Testing strategy used

- **Unit-level checks** for service/repository rules (status transitions, inventory update guards, role constraints).
- **API-level validation** for endpoint payloads and expected response behavior ('200', '400', '404').
- **Workflow testing** for end-to-end request lifecycle:
 1. Seller creates request.
 2. Distributor fulfills/escalates.
 3. Manufacturer dispatches when applicable.
 4. Receiver confirms delivery.
 5. Inventory updates after receive.
- **Regression testing** for previously reported failures (signup + missing 'Email' column, invalid order transitions).

3.2 Debugging evidence to demonstrate

- Reproduce issue with known input.
- Capture logs/stack trace and identify failing module.
- Apply fix in isolated layer (service/repository/controller).
- Retest the same scenario + edge cases.
- Record before/after behavior.

3.3 Suggested test cases table

1. Signup on legacy DB without 'Users.Email' column -> should not crash.
2. Distributor fulfill with insufficient stock -> should reject transition.
3. Manufacturer dispatch with insufficient stock -> should reject transition.
4. Seller receive before 'OnTheWayToSeller' -> should reject.
5. Seller receive after 'OnTheWayToSeller' -> should update seller inventory.
6. Distributor receive before 'OnTheWayToDistributor' -> should reject.
7. Distributor receive after 'OnTheWayToDistributor' -> should update distributor inventory.
8. Invalid role on incoming/outgoing endpoints -> should return '400'.

3.4 What to include in your report/demo

- Test matrix with expected vs actual results.
- Screenshots of successful and failed validation paths.
- Bug history (issue -> root cause -> fix -> retest).
- Coverage note: what is tested and what is not.

4) Suitable Deployment Techniques (LO4)

4.1 Option A: Traditional Windows Server + IIS + SQL Server

Use when: simple academic deployment and low ops complexity.

Pros:

- Easy for ASP.NET MVC/Web API projects.
- Familiar setup for .NET Framework projects.

- Straightforward demo environment.

****Cons:****

- Manual scaling and environment setup.
- Harder to keep environments identical.

4.2 Option B: Dockerized services

****Use when:**** you need repeatable environments and cleaner delivery pipeline.

****Pros:****

- Consistent runtime from dev to production.
- Easier CI/CD integration.
- Better portability and rollback.

****Cons:****

- Additional container learning curve.
- .NET Framework apps may require Windows containers and more setup care.

4.3 Option C: Kubernetes orchestration

****Use when:**** high-scale, multi-service, production-grade operations are required.

****Pros:****

- Auto-scaling, self-healing, rolling deployments.
- Strong service management for larger SOC systems.

****Cons:****

- Highest operational complexity.
- Overkill for small/medium coursework projects.

4.4 Recommended approach for this project

- ****Primary recommendation (assignment fit):**** IIS + SQL Server for baseline demonstration.
- ****Forward-looking recommendation:**** add Docker-based packaging for reproducibility and future scalability.
- Document release steps, configuration strategy, backup policy, and rollback plan.

5) Conclusion / Evaluation Summary

- The Cozy Comfort solution aligns with SOC intent by separating API services and client consumption.
- To score higher, focus on:
 1. Strong test evidence (automated + manual traceability).
 2. Clear deployment justification with trade-off analysis.
 3. Professional diagrams + maintainability rationale.
- Final report should explicitly map each deliverable to LO2/LO3/LO4 and rubric criteria.